

Global Exception Handling & Production Error Contracts

Complete detailed study notes • Beginner-friendly • Book Catalog project

DAY 5 GOAL

Understand what happens when something goes wrong in a Spring Boot API, how to create meaningful exceptions, handle them in one central place, return consistent JSON, handle validation failures, and keep internal technical details away from clients.

What you will learn

1. Custom exceptions and why they are useful
2. Central error handling with `@RestControllerAdvice` and `@ExceptionHandler`
3. Stable error responses and application error codes
4. Validation failures and field-level messages
5. Correct HTTP status codes
6. Safe logging and protection from stack-trace leaks
7. How to implement and test everything in the Book Catalog

CORE FLOW

Request → Controller → Service → problem occurs → exception → global handler → `ApiError` → HTTP response.
Detailed technical information goes to server logs, not to the client.

Learning rule: understand the problem first; memorize Spring annotation names second.

INDEX — COMPLETE TOPICS

The notes continue through all 32 numbered topics. Headings flow naturally across pages; they are not forced onto separate pages.

1. What is an exception?
2. What happens when an exception is not handled?
3. Why APIs need a common error format
4. Custom exceptions
5. `ResourceNotFoundException`
6. `DuplicateResourceException`
7. Business errors vs technical failures
8. `@RestControllerAdvice`
9. `@ExceptionHandler`
10. How the global handler works
11. Designing the error response
12. Timestamp, status, code, message, path
13. Validation details
14. Application error codes
15. HTTP status codes
16. 400 Bad Request
17. 404 Not Found
18. 409 Conflict
19. 500 Internal Server Error
20. Why stack traces should not reach clients
21. Safe logging
22. Building `ApiError`
23. Building `ValidationError`
24. Handling validation failures
25. Handling unexpected exceptions
26. Client response vs server logs
27. Complete Book Catalog implementation
28. Testing every important failure path
29. Common beginner mistakes
30. Production-minded improvements
31. Practice questions
32. Final self-check

TARGET OUTCOME

You can look at an API failure and explain what happened, which exception represents it, which HTTP status is appropriate, what JSON the client receives, and what should be logged internally.

1 What is an exception?

An **exception** is Java's way of saying that the normal path of the program cannot continue as expected.

For example, the Book Catalog receives a request for book ID 50, but there is no book 50. Your code needs a way to say: "The requested book does not exist."

```
Book book = repository.findById(id);

if (book == null) {
    throw new RuntimeException("Book not found");
}
```

The idea is correct, but `RuntimeException` is too generic for a real API. We want an exception whose name explains the actual problem.

SIMPLE IDEA

Think of an exception as an alarm. The code that notices the problem raises the alarm; another part of the application decides how to respond.

2 What happens when an exception is not handled?

If an exception is not handled by your application, Spring eventually produces an error response. The exact response depends on the exception and configuration.

The important point is that you want **your API** to control the format and meaning of errors instead of letting every failure look different.

```
Service throws exception
↓
No application-specific handler
↓
Spring handles the failure
↓
Client receives a generic error
```

For known failures such as "book not found" or "duplicate ISBN", we want predictable responses.

3 Why APIs need a common error format

Imagine three endpoints return three different formats. A frontend now has to understand a different structure for every endpoint.

```
GET /books/50
{ "message": "Not found" }

POST /books
{ "error": "duplicate" }

PUT /books/50
{ "status": 409, "reason": "Conflict" }
```

A common error format makes the API easier for frontend developers, mobile clients, tests and future developers to use.

A typical contract can contain timestamp, status, code, message, path and optional validation details.

RULE

The exact field names are your design choice. Consistency is the important part.

4 Custom exceptions

A custom exception is simply a Java class you create for a situation that matters to your application.

```
public class ResourceNotFoundException
    extends RuntimeException {

    public ResourceNotFoundException(String message) {
        super(message);
    }
}
```

Now the Service can throw an exception whose name already explains the problem.

```
throw new ResourceNotFoundException(
    "Book not found with id: " + id
);
```

The global handler can recognize this exception and turn it into the correct HTTP response.

5 ResourceNotFoundException

Use this when the client asks for something that does not exist.

Examples:

- GET /api/books/99 when book 99 does not exist.
- PUT /api/books/99 when book 99 does not exist.
- DELETE /api/books/99 when book 99 does not exist.

```
public Book getBook(Long id) {
    return repository.findById(id)
        .orElseThrow(() ->
            new ResourceNotFoundException(
                "Book not found with id: " + id
            )
        );
}
```

The usual HTTP status is **404 Not Found**.

- GET /api/books/99 and book 99 does not exist.
- PUT /api/books/99 when book 99 does not exist.
- DELETE /api/books/99 when book 99 does not exist.

```
public Book getBook(Long id) {
    return repository.findById(id)
        .orElseThrow(() ->
            new ResourceNotFoundException(
                "Book not found with id: " + id
            )
        );
}
```

The usual HTTP status is **404 Not Found**.

6 DuplicateResourceException

Use this when the request is valid but conflicts with something that already exists.

For the Book Catalog, assume ISBN is unique. If a user tries to create a second book with the same ISBN, the operation conflicts with the current data.

```
if (repository.existsByIsbn(request.isbn())) {
    throw new DuplicateResourceException(
        "A book with this ISBN already exists"
    );
}
```

A common HTTP status for this is **409 Conflict**.

7 Business errors vs technical failures

Business error	Technical failure
Book does not exist.	Database connection fails.
ISBN already exists.	Unexpected NullPointerException.
Operation violates a business rule.	External service times out.
Request data fails validation.	Server configuration is broken.

Business errors are situations your application understands and can usually explain to the user. Technical failures are internal problems that developers need to investigate.

The client often needs a useful business-level explanation, while the developer may need a stack trace and technical context.

Think like a production developer

Do not expose your internal implementation just because the server knows it. Return what the client needs and log what the development team needs.

8 @RestControllerAdvice

@RestControllerAdvice marks a class as a central place for handling errors from REST Controllers.

```
@RestControllerAdvice
public class GlobalExceptionHandler {
}
```

Instead of writing similar error-handling code in every Controller, you can keep common REST error handling in one place.

```
BookController
├──
│   GlobalExceptionHandler
│   └──
│       AuthorController
```

This gives your API one consistent place to decide how exceptions become HTTP responses.

9 @ExceptionHandler

@ExceptionHandler connects an exception type to the method that should handle it.

```
@ExceptionHandler(ResourceNotFoundException.class)
public ResponseEntity<ApiError> handleNotFound(
    ResourceNotFoundException ex,
    HttpServletRequest request) {
    // build response
}
```

Annotation	Simple meaning
@RestControllerAdvice	This class handles REST errors centrally.
@ExceptionHandler	This method handles a particular exception type.

10 How the global handler works

Suppose the Service throws ResourceNotFoundException. You do not want the Service to know about HTTP status codes or JSON response formatting.

```
Controller
┆
┆   ↓
┆   Service
┆   ↓
┆   throw ResourceNotFoundException
┆   ↓
┆   GlobalExceptionHandler
┆   ↓
┆   create ApiError
┆   ↓
┆   return HTTP 404 JSON
```

This keeps responsibilities separated: the Service knows the business rule; the handler knows how that failure should look over HTTP.

11 Designing the error response

A good error response is predictable. A client should be able to look for the same fields whether the error came from a missing book, duplicate ISBN or validation failure.

```
{
  "timestamp": "2026-09-08T10:30:00Z",
  "status": 404,
  "code": "BOOK_NOT_FOUND",
  "message": "Book not found",
  "path": "/api/books/50",
  "validationErrors": []
}
```

You can add more fields later if your application genuinely needs them, but avoid changing the contract casually.

12 Timestamp, status, code, message, path

Field	Meaning
timestamp	When the error response was created.
status	HTTP status such as 400, 404, 409 or 500.
code	Stable application-specific identifier.
message	Human-readable explanation.
path	Endpoint that produced the error.

These fields answer five basic questions: **When? What HTTP result? What kind of application error? What happened? Where did it happen?**

13 Validation details

Validation is different because one request can contain multiple problems.

```
{
  "title": "",
  "author": "A",
  "price": -20
}

{
  "status": 400,
  "code": "VALIDATION_FAILED",
  "message": "Request validation failed",
  "path": "/api/books",
  "validationErrors": [
    {"field": "title", "message": "must not be blank"},
    {"field": "price", "message": "must be greater than 0"}
  ]
}
```

Field-level errors let the client show the user exactly what needs to be corrected.

14 Application error codes

Do not rely only on human-readable messages. A message might change because you improve wording or support another language.

```
BOOK_NOT_FOUND
BOOK_ALREADY_EXISTS
VALIDATION_FAILED
INTERNAL_SERVER_ERROR
```

The code gives the client a stable identifier it can use for program logic.

Good practice

Keep codes clear and stable. Do not create a new code every time you slightly change the message.

15 HTTP status codes

Status	Use it when
400 Bad Request	The request is invalid, often because validation failed.
401 Unauthorized	Authentication is missing or invalid.

Status	Use it when
403 Forbidden	The client is authenticated but not allowed to perform the action.
404 Not Found	The requested resource does not exist.
409 Conflict	The request conflicts with the current resource state.
500 Internal Server Error	An unexpected server-side failure occurred.

16 400 Bad Request

Use 400 when the client sent a request that the API cannot accept as valid input.

- Required field is missing.
- Title is blank.
- Price is negative when your rule says it must be positive.
- ISBN format fails validation.

The important distinction is that the request itself is invalid, rather than the server simply being unable to find a resource.

17 404 Not Found

404 means the requested resource could not be found.

```
GET /api/books/50

Database:
50 → does not exist

Response:
404 Not Found
```

Do not return 500 just because the lookup returned nothing. A missing book can be a normal, expected business situation.

18 409 Conflict

409 is useful when the request conflicts with the current state of the resource or system.

```
POST /api/books

ISBN: 9781234567890
↓
ISBN already belongs to another book
↓
409 Conflict
```

Situation	Likely status
Invalid ISBN format	400
ISBN already exists	409
Book ID does not exist	404

19 500 Internal Server Error

500 is the safety net for unexpected server-side failures.

Example: a programming bug causes an unexpected exception, or a technical dependency fails in a way your application cannot handle gracefully.

```
Unexpected technical failure
↓
log detailed exception
↓
return safe 500 response
```

The client does not need a Java stack trace. It needs to know that the server could not complete the request.

20 Why stack traces should not reach clients

A stack trace is useful to developers but can expose internal details such as class names, package structure, file paths, SQL details or library information.

```
BAD:
{
  "error": "java.lang.NullPointerException",
  "stackTrace": "com.example.service.BookService..."
}
```

```
BETTER:
{
  "status": 500,
  "code": "INTERNAL_SERVER_ERROR",
  "message": "An unexpected error occurred",
  "path": "/api/books"
}
```

Golden rule

Detailed technical information belongs in protected server logs, not in the public API response.

21 Safe logging

Logging is how you keep technical details available for debugging without sending them to the client.

```
log.error(
    "Unexpected error while creating book",
    ex
);
```

For known business failures, you may not need a full stack trace at error level. For unexpected failures, logging the exception and stack trace is often important.

Never intentionally log passwords, access tokens, API keys or other secrets.

22 Building ApiError

A simple Java record can represent the common response.

```
public record ApiError(
    Instant timestamp,
    int status,
    String code,
    String message,
    String path,
    List<ValidationError> validationErrors
) {}
```

The response object is not the exception. The exception describes what went wrong inside the application; `ApiError` describes what you want to send outside the application.

23 Building ValidationError

Validation errors can be represented separately so that each invalid field has its own message.

```
public record ValidationError(  
    String field,  
    String message  
) {}
```

Then `ApiError` can contain a list of these objects.

```
List<ValidationError> validationErrors
```

24

Handling validation failures

When you use Jakarta Bean Validation on a request DTO, Spring can detect invalid input before your normal business logic runs.

```
Request  
↓  
DTO validation  
↓  
validation failure  
↓  
Global Exception Handler  
↓  
400 VALIDATION_FAILED  
↓  
field-level errors
```

Your handler's job is to extract the field names and validation messages and place them into the standard response.

Do not memorize blindly

The exact Spring exception involved depends on how validation is being used. Understand the flow first, then learn the specific exception type in your project.

25

Handling unexpected exceptions

Specific exceptions should get specific responses. A final generic handler can catch unexpected exceptions so that clients still receive your standard error format.

```
@ExceptionHandler(Exception.class)  
public ResponseEntity<ApiError> handleUnexpected(  
    Exception ex,  
    HttpServletRequest request) {  
  
    log.error("Unexpected server error", ex);  
  
    ApiError error = new ApiError(  
        Instant.now(),  
        500,  
        "INTERNAL_SERVER_ERROR",  
        "An unexpected error occurred",  
        request.getRequestURI(),  
        List.of()  
    );  
  
    return ResponseEntity  
        .status(HttpStatus.INTERNAL_SERVER_ERROR)  
        .body(error);  
}
```

The generic handler should be a safety net, not an excuse to treat every known business failure as a 500.

26

Client response vs server logs

Client should receive	Server should log
HTTP status	Exception type
Stable error code	Stack trace when useful
Safe message	Technical context
Request path	Debug information

The client and the developer are solving different problems. The client needs a stable contract. The developer needs enough detail to diagnose the failure.

27 Complete Book Catalog implementation

Step 1 — Create exceptions

```
public class ResourceNotFoundException
    extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }
}

public class DuplicateResourceException
    extends RuntimeException {
    public DuplicateResourceException(String message) {
        super(message);
    }
}
```

Step 2 — Create response models

```
public record ValidationError(
    String field,
    String message
) {}

public record ApiError(
    Instant timestamp,
    int status,
    String code,
    String message,
    String path,
    List<ValidationError> validationErrors
) {}
```

Step 3 — Create the global handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    // ResourceNotFoundException → 404
    // DuplicateResourceException → 409
    // validation failure → 400
    // unexpected Exception → 500
}
```

Step 4 — Throw meaningful exceptions from the Service

```
return repository.findById(id)
    .orElseThrow(() ->
        new ResourceNotFoundException(
            "Book not found with id: " + id
        )
    );
```

Step 5 — Test every failure path

Do not stop when the application starts. The real goal is to prove that the API behaves correctly when things go wrong.

28 Testing every important failure path

Test case	Expected result
GET existing book	Normal success response.
GET missing book	404 + BOOK_NOT_FOUND/resource-not-found error.
Create duplicate ISBN	409 + duplicate/conflict error.
Blank title	400 + title validation error.
Invalid price	400 + price validation error.
Unexpected server failure	500 + safe message, no stack trace.

For each test, verify both the **HTTP status** and the important **JSON fields**. Do not test only that “an error happened.”

29 Common beginner mistakes

Mistake	Better approach
Returning a different JSON format for every endpoint	Use one common error structure.
Using RuntimeException for every business problem	Create meaningful custom exceptions.
Returning stack traces	Log technical details on the server.
Catching everything inside every Controller	Centralize common REST error handling.
Using 500 when a book is missing	Use 404.
Using 400 for every error	Choose the status based on what happened.
Logging secrets	Keep passwords and tokens out of logs.

30 Production-minded improvements

- Keep application error codes stable.
- Keep the JSON shape consistent across endpoints.
- Automatically include the request path.
- Return useful field-level validation information.
- Log unexpected technical failures with enough context.
- Never expose stack traces or secrets.
- Test status codes and important response fields.
- Document the error contract in the README.
- Keep business rules in Services rather than inside the global handler.
- Avoid putting database or internal implementation details into public error messages.

Day 5 success condition

You can intentionally cause a missing-book error, duplicate-ISBN error, validation error and unexpected server error, and you can predict exactly what the client should receive for each one.

31. PRACTICE QUESTIONS

Answer these without looking at the notes. Say your answers aloud and then implement the coding questions in your Book Catalog.

1.	What is an exception in Java?
2.	Why is a generic RuntimeException not ideal for every business failure?
3.	What problem does global exception handling solve?
4.	What is the purpose of @RestControllerAdvice?
5.	What does @ExceptionHandler do?
6.	When would you throw ResourceNotFoundException?
7.	When would you throw DuplicateResourceException?
8.	What is the difference between a business error and a technical failure?
9.	Why should an API have a stable error code?
10.	What should timestamp, status, code, message and path represent?
11.	Why are validation errors special?
12.	When should an API return 400?
13.	When should an API return 404?
14.	When should an API return 409?
15.	When should an API return 500?
16.	Why should a stack trace not be returned to the client?
17.	Where should detailed technical exception information go?
18.	What is the difference between the client response and server logs?
19.	Why should the Service not build the HTTP error response itself?
20.	How would you handle a duplicate ISBN?
21.	How would you represent multiple validation errors?
22.	What should happen when an unexpected NullPointerException occurs?
23.	Why is a generic Exception handler useful?

- | | |
|-----|---|
| 24. | What should you test for a missing book? |
| 25. | Explain the complete error flow from HTTP request to error response. |
| 26. | Build the complete global exception handling system in your Book Catalog without copying the notes. |

32. FINAL SELF-CHECK

Before moving to Day 6

- I can explain exceptions in simple words.
- I can explain why global exception handling is useful.
- I created `ResourceNotFoundException` and `DuplicateResourceException`.
- I understand `@RestControllerAdvice` and `@ExceptionHandler`.
- My API returns a consistent error JSON structure.
- I know the difference between 400, 404, 409 and 500.
- Validation errors include useful field-level information.
- Stack traces are not exposed to clients.
- Unexpected failures are logged safely.
- I wrote tests for the important failure paths.
- I updated the README and committed my work.

Final challenge

Delete or temporarily break one piece of your Book Catalog code and observe the failure. Then explain the entire journey: request → controller → service → exception → global handler → `ApiError` → HTTP response → server log.